

//BFS 广度优先遍历 层次遍历

```
BiTree FindMaxNode_BFS(BiTree T) {
    if (T == NULL) return NULL;

    C_Queue Q;          // 循环队列
    BiTree p;           // 存放队头元素

    InitQueue(Q);
    EnQueue(Q, T); // 根节点入队

    while (!QueueEmpty(Q)) {

        DeQueue(Q, p); // 队头元素出队

        // 执行需要的操作，如打印输出 p->data 等

        // 2. 将子节点入队
        if (p->lchild != NULL) {
            EnQueue(Q, p->lchild);
        }
        if (p->rchild != NULL) {
            EnQueue(Q, p->rchild);
        }
    }
}
```

//深度优先遍历

```
void DFS(BiTree T){
    if(T==NULL) return;

    Stack s;
    InitStack(s); // 初始化栈

    BiTree p; // 临时存放栈头结点
    push(s,p);

    while(!StackEmpty(s)){
        pop(s,p); // 栈首结点出栈

        // 执行需要的操作
```

```

        //孩子结点入栈
        if(p->lchild!=NULL){
            push(s,p->lchild);
        }
        if(p->rchild!=NULL){
            push(s,p->rchild);
        }
    }
}

```

//求二叉树中叶子节点的数量

```

int CountLeaf(BiTree T){
    if(T==NULL) return 0; //空节点不是叶子节点
    if(!T->left&&!T->right){
        return 1;          //如果是叶子节点则返回 1
    }
    return CountLeaf(T->left)+CountLeaf(T->right) //分治思想，分别递归左子树和右子
    树，然后相加
}

```

//求二叉树的深度

```

int Getdepth(BiTree &T){
    if(T==NULL) return 0;
    int left = Getdepth(T->lchild);
    int right = Getdepth(T->rchild);
    return (left>right?left:right)+1;
}

```

//求二叉树中值为 x 的层号

```

int FindLevel(BiTree T,int depth,int x){
    if(T==NULL) return 0;
    if(T->data==x){
        return depth;
    }
    //递归查找左子树
}

```

```

    int left = FindLevel(T->lchild,depth+1,x);
    if(left!=0){
        return left;
    }
    return FindLevel(T->rchild,depth+1,x);
}

//void BFS(BiTree T){
    Queue&q;
    q.rear=0;
    q.front=0; //初始化队列
    q.data[q.rear++] = T; //根结点入队
    while(q.rear!=q.front){
        BNode * p =q.data[q.front++]; //保存当前首结点
        int value = p->data;
        printf("%d",value);
        if(p->lchild!=NULL){
            q.data[q.rear++] = p->lchild; //左子结点入队
        }
        if(p->rchild!=NULL){
            q.data[q.rear++] = p->rchild; //右子结点入队
        }
    }
}
}

```

//计算二叉树中所有结点的个数

```

int Count(BiTree T){
    if (T == NULL) {
        return 0; // 空树返回 0
    }
    BNode*q;
    Queue Q;
    InitQueue(Q); //初始化队列
    int n=0; //计数器
    EnQueue(Q, T); //根结点入队
    while(!QueueEmpty(Q)){
        //TODO
        DeQueue(Q,q); //队头结点出队
        n++;
        if(q->lchild!=NULL){
            EnQueue(Q, q->lchild);
        }
    }
}

```

```

        if(q->rchild!=NULL){
            EnQueue(Q, q->rchild);
        }
    }
    return n;
}

```

//计算二叉树中所有叶子结点的个数

```

int Count_leaf(BiTree T){
    if (T == NULL) {
        return 0; // 空树返回 0
    }
    Queue Q;
    BNode*q;
    InitQueue(Q);//初始化队列
    int n=0;//计数器
    EnQueue(Q, T);//根结点入队
    while(!QueueEmpty(Q)){
        //TODO
        DeQueue(Q,q)//队头结点出队
        if(!q->lchild&&!q->rchild){
            n++; //没有左子树也没有右子树的结点为叶结点，记录叶结点的个数
        }

        if(q->lchild!=NULL){
            EnQueue(Q, q->lchild);
        }
        if(q->rchild!=NULL){
            EnQueue(Q, q->rchild);
        }
    }
    return n;
}

```

//求二叉树中值为 x 的层号

//假设队列支持 size()操作，返回当前层次的所有结点个数

```

int Find_x(BiTree T){
    if(T==NULL) return 0;
    Queue Q;
    InitQueue(Q);//初始化数组
    BNode*q;
    int level = 1;
    EnQueue(Q,T);//根结点入队

```

```

while(!QueueEmpty(Q)){
    int levelSize = Q.size(); //返回当前层次的所有结点的个数
    for(int i=0;i<levelSize;i++){
        DeQueue(Q,q); //出队当前层结点
        if(q->data==x){
            return level; //返回当前层数
        }
        if(q->lchild!=NULL){
            EnQueue(Q,q->lchild); //左孩子入队
        }
        if(q->rchild!=NULL){
            EnQueue(Q,q->rchild); //右孩子入队
        }
    }
    level ++; //进入下一层，层数 + 1
}
return 0; //若没有找到 x，则返回 0
}

```

//求二叉树的最大深度

```

int max_depth(BiTree T){
    if(T==NULL) return 0;
    Queue Q;
    InitQueue(Q);
    EnQueue(Q,T);
    BNode*q;
    int level = 0;
    while(!QueueEmpty(Q)){
        level++;
        for(int i=0;i<Queue.size();i++){
            DeQueue(Q,q);
            if(q->lchild!=NULL){
                EnQueue(Q,q->lchild);
            }
            if(p->rchild!=NULL){
                EnQueue(Q,q->rchild);
            }
        }
    }
    return level;
}

```

//如果不能用 size 函数

```

// 假设 NULL_NODE 是一个特殊的、不属于树的标记指针
Constant NULL_NODE = NULL

// 函数名: MaxDepth_BFS_Marker
int MaxDepth_BFS_Marker(BiTree T) {
    if (T == NULL) return 0;

    C_Queue Q;          // 循环队列
    BiTree p;
    int depth = 0;

    InitQueue(Q);
    EnQueue(Q, T);      // 根节点入队
    EnQueue(Q, NULL_NODE); // 【标记】根节点后面加入层标记

    while (!QueueEmpty(Q)) {
        DeQueue(Q, p);

        // 1. 遇到层标记
        if (p == NULL_NODE) {
            depth++; // 【关键】处理完一层，深度加 1

            // 检查：如果队列中还有元素，说明下一层已经入队
            if (!QueueEmpty(Q)) {
                EnQueue(Q, NULL_NODE); // 【新的标记】在下一层末尾再加入一个标记
            }
            // 否则，队列为空，树遍历完毕，退出循环
        }

        // 2. 遇到正常节点
        else {
            // 正常访问和入队子节点
            if (p->lchild != NULL) {
                EnQueue(Q, p->lchild); // 子节点入队
            }
            if (p->rchild != NULL) {
                EnQueue(Q, p->rchild); // 子节点入队
            }
        }
    }

    return depth; // 返回最大深度
}

```

// 求二叉树的最大深度:递归法 (DFS)

```
int max_depth_dfs(BiTree T) {
    if (T == NULL) {
        return 0; // 空树深度为 0
    }
    // 递归求左右子树的深度
    int left_depth = max_depth_dfs(T->lchild);
    int right_depth = max_depth_dfs(T->rchild);

    // 当前树的深度 = 左右子树深度的最大值 + 1 (即加上根节点这一层)
    return (left_depth > right_depth ? left_depth : right_depth) + 1;
}
```

//找出二叉树中值最大的结点

```
// 函数名: FindMaxNode_BFS
// 功能: 使用循环队列进行层序遍历, 找出值最大的节点
// 返回值: 值最大的节点的指针
BiTree FindMaxNode_BFS(BiTree T) {
    if (T == NULL) return NULL;

    C_Queue Q;          // 循环队列
    BiTree p;

    BiTree MaxNode = T; // 记录最大值节点的指针, 初始指向根节点
    int MaxVal = T->data; // 记录当前的最大值

    InitQueue(Q);
    EnQueue(Q, T);

    while (!QueueEmpty(Q)) {

        DeQueue(Q, p); // 队头元素出队

        // 1. 访问节点: 检查并更新最大值
        if (p->data > MaxVal) {
            MaxVal = p->data;
            MaxNode = p;
        }
    }
}
```

```

        // 2. 将子节点入队
        if (p->lchild != NULL) {
            EnQueue(Q, p->lchild);
        }
        if (p->rchild != NULL) {
            EnQueue(Q, p->rchild);
        }
    }

    return MaxNode;
}

// 假设全局变量或通过主函数调用时传入一个引用/指针
Type BiTree: 指向 BiTNode 的指针
BiTree MaxNode = NULL; // 用于记录当前找到的值最大的节点指针

// 函数名: FindMaxNode_Recursive
// 功能: 递归遍历二叉树, 更新全局最大节点指针
void FindMaxNode_Recursive(BiTree T) {
    if (T == NULL) {
        return; // 递归终止条件
    }

    // 1. 访问当前节点 (根)
    // 检查并更新 MaxNode
    if (MaxNode == NULL || T->data > MaxNode->data) {
        MaxNode = T;
    }

    // 2. 递归遍历左子树
    FindMaxNode_Recursive(T->lchild);

    // 3. 递归遍历右子树
    FindMaxNode_Recursive(T->rchild);
}

```


//查找二叉树中 data 域等于 key 的结点是否存在，若存在，将 q 指向它，否则 q 为空

```
void Find_key(BiTree T,int key,BiTree *q){
    *q = NULL;
    if(T==NULL) return;
    BiTree p;
    Queue Q;
    InitQueue(Q); //初始化队列
    EnQueue(Q,T); //根节点入队
    while(!QueueEmpty(Q)){
        DeQueue(Q,p);
        if(p->data==key){
            *q = p;
            return;
        }
        if(p->lchild!=NULL){
            EnQueue(Q,p->lchild);
        }
        if(p->rchild!=NULL){
            EnQueue(Q,p->rchild);
        }
    }
}
```

//输出先序遍历第 k 个结点的值

//递归实现

int count = 0; //全局变量

void preOrder(BiTree T,int k){

if(T==NULL) return;

count++;

if(count==k){

printf("%d",T->data);

return; //问题出在找到目标结点后，虽然执行了 return;，但这个 return

//只会退出当前这一层的递归调用，而不会阻止上一层或更上层的递归

调用继续执行

}

if(T->lchild){

preOrder(T->lchild,k);

}

if(T->rchild){

preOrder(T->rchild,k);

}

```
}
```

正解:

```
int count = 0;
```

```
int preOrder(BiTree T,int k){
```

```
    if(T==NULL) return 0;
```

```
    count++;
```

```
    if(count==k){
```

```
        printf("%d",T->data);
```

```
        return 1;//向上通知已经找到目标
```

```
    }
```

```
    if(preOrder(T->lchild,k)){
```

```
        return 1; //相当于接收通知， 如果已经找到了， 直接返回 1
```

```
    }
```

```
    if(preOrder(T->rchild,k)){
```

```
        return 1; //相当于接收通知， 如果已经找到了， 直接返回 1
```

```
    }
```

```
}
```

//输出先序遍历第 k 个结点的值

//非递归实现 => 栈

//需要明确的是，用栈进行先序遍历，由于是根左右的顺序，因此根先入队然后出队后，应该先让右结点入队

```
void preOrder(BiTree T,int k){
```

```
    if(T==NULL||k<0) return ;
```

```
    Stack s;
```

```
    InitStack(s); //初始化栈
```

```
    push(s,T); //根节点入栈
```

```
    BiTree p;
```

```
    int count = 0; //计数器
```

```
    while(!StackEmpty(s)){
```

```
        pop(s,p);
```

```
        count++;
```

```
        if(count==k){
```

```
            printf("%d",p->data);
```

```
            return;
```

```
        }
```

```
        if(p->rchild){
```

```
            push(s,p->rchild); //右结点入栈
```

```
        }
```

```

        if(p->lchild){
            push(s,p->lchild); //左结点入栈
        }
    }
}

```

//把二叉树所有结点的左右子树交换

//递归

```

void Mirror(BiTree T){
    if(T==NULL) return;

```

//核心操作代码

```

    BiTree temp = T->lchild;

```

```

    T->lchild = T->rchild;

```

```

    T->rchild = temp;

```

//递归

```

    Mirror(T->lchild);

```

```

    Mirror(T->rchild);

```

```

}

```

//非递归

```

void Mirror(BiTree T){
    if(T==NULL) return;
    BiTree p;
    Queue Q;
    InitQueue(Q);
    EnQueue(Q,T);

```

```

    while(!QueueEmpty(Q)){
        DeQueue(Q,p);

```

```

        BiTree temp = p->lchild;

```

```

        p->lchild = p->rchild;

```

```

        p->rchild = temp;

```

```

        if(p->lchild){

```

```

            EnQueue(Q,p->lchild);

```

```

        }

```

```

        if(p->rchild){

```

```

            EnQueue(Q,p->rchild);

```

```

        }

```

```

    }

```

```

}

```

//判断二叉树是不是正则二叉树(即每个结点的度均为 0 或 2)

```
bool isZhengze(BiTree T){
    if(T==NULL) return 0;
    Queue Q;
    InitQueue(Q);
    EnQueue(Q,T);
    BiTree q;
    while(!QueueEmpty(Q)){
        DeQueue(Q,q);

        if((q->lchild==NULL&&q->rchild!=NULL)||((q->lchild!=NULL&&q->rchild==NULL)){
            return 0;
        }
        if(q->lchild){
            EnQueue(Q,q->lchild);
        }
        if(q->rchild){
            EnQueue(Q,q->rchild);
        }
    }
    return 1;
}
```

// 递归判断 (假设空树也返回 1)

```
bool isStrictlyBinaryRecursive(BiTree T) {
    if (T == NULL) {
        return 1;
    }

    // 检查当前节点 T
    // 只有左孩子，没有右孩子 -> 度为 1，返回 0
    if (T->lchild != NULL && T->rchild == NULL) {
        return 0;
    }
    // 只有右孩子，没有左孩子 -> 度为 1，返回 0
    if (T->lchild == NULL && T->rchild != NULL) {
        return 0;
    }

    // 递归检查左右子树
    return isStrictlyBinaryRecursive(T->lchild) && isStrictlyBinaryRecursive(T->rchild);
}
```

//先序非递归遍历二叉树

//栈实现

```
void PreOrder(BiTree T){
    if(T==NULL) return;
    Stack s;
    InitStack(s);
    push(s,T);//根节点入栈
    BiTree q;
    while(!StackEmpty(s)){
        pop(s,q);
        printf("%d",q->data);
        if(q->rchild){
            push(s,q->rchild);
        }
        if(q->lchild){
            push(s,q->lchild);
        }
    }
}
```

//中序遍历

//栈+工作指针

```
void Inorder(BiTree T){
    if(T==NULL) return;
    Stack s;
    InitStack(s);
    push(s,T);
    BiTree p;
    while(p!=NULL||!StackEmpty(s)){
        while(p!=NULL){
            push(s,p);
            p=p->lchild;
        }
        pop(s,p);
        printf("%d",p->data);

        push(s,p->rchild);
    }
}
```

//后序非递归遍历二叉树

//左右根相当于根右左的逆序，因此后序可以看作是先序遍历的变形，只不过先序遍历是先右子树入栈，先左子树出栈

//而根右左则让左子树先入，右子树先出

//双栈实现

```
void backorder(BiTree T){
    if(T==NULL) return;
    Stack s1,s2;
    InitStack(s1);
    InitStack(s2);
    BiTree p;
    push(s1,T);
    while(!StackEmpty(s1)){
        pop(s1,p);
        if(p->lchild){
            push(s1,p->lchild);
        }
        if(p->rchild){
            push(s1,p->rchild);
        }
        push(s2,p);
    }
    while(!StackEmpty(s2)){
        pop(s2,p);
        printf("%d",p->data);
    }
}
```